

Guide de Preparation

Entretien Technique Data Engineer

Profil 2 a 4 ans d'experience

Plus de 150 questions-reponses classees par theme

SQL – Python – ETL/ELT – Data Modeling – Spark/PySpark
Kafka – Airflow – Data Warehouse – Cloud – System Design

A utiliser comme support de revision active : cachez la reponse et essayez de la reformuler a voix haute avant de la lire.

Fiche de preparation personnelle

Table des matières

1	SQL	2
1.1	Bases : SELECT, WHERE, DISTINCT	2
1.2	GROUP BY et agregations	2
1.3	JOIN	2
1.4	Window Functions	3
1.5	CTE et sous-requetes	4
1.6	Optimisation SQL	4
2	Python	6
3	ETL / ELT	9
4	Data Modeling	11
5	Apache Spark / PySpark	13
6	Kafka	15
7	Airflow	17
8	Data Warehouse et formats de table	19
9	Cloud (AWS, Azure, GCP)	21
9.1	AWS	21
9.2	Azure	21
9.3	GCP	21
10	System Design Data, Data Quality et Orchestration	22
10.1	System Design	22
10.2	Data Quality	23
10.3	Orchestration	23
11	Questions sur vos projets	25
12	Scenarios de production	27

1. SQL

1.1 Bases : SELECT, WHERE, DISTINCT

Q.

A quoi servent **DISTINCT** et **LIMIT**, et dans quel ordre les clauses SQL sont-elles réellement exécutées ?

Reponse attendue

DISTINCT élimine les doublons du résultat final ; **LIMIT** restreint le nombre de lignes retournées. Piège classique d'entretien : l'ordre d'écriture (**SELECT ... FROM ... WHERE ... GROUP BY ... HAVING ... ORDER BY ... LIMIT**) n'est *pas* l'ordre d'exécution réel. Le moteur exécute dans l'ordre : **FROM** → **WHERE** → **GROUP BY** → **HAVING** → **SELECT** → **DISTINCT** → **ORDER BY** → **LIMIT**. C'est pourquoi **WHERE** ne peut pas utiliser un alias défini dans **SELECT**, ni une fonction d'agregation.

1.2 GROUP BY et agregations

Q.

Quelle est la différence entre **WHERE** et **HAVING** ?

Reponse attendue

WHERE filtre les lignes individuelles *avant* l'agregation et ne peut pas utiliser **COUNT()**, **SUM()**, etc. **HAVING** filtre les groupes *après* l'agregation par **GROUP BY** et peut donc utiliser des fonctions d'agregation (ex : **HAVING COUNT(*) > 10**).

```
1 SELECT department, COUNT(*) AS nb_employees, AVG(salary) AS salaire_moyen
2 FROM employees
3 GROUP BY department
4 HAVING COUNT(*) > 5
5 ORDER BY salaire_moyen DESC;
```

1.3 JOIN

Q.

Quelles sont les différences entre **INNER JOIN**, **LEFT JOIN**, **RIGHT JOIN**, **FULL JOIN** et **SELF JOIN** ?

Reponse attendue

INNER JOIN : uniquement les lignes avec correspondance dans les deux tables. **LEFT JOIN** : toutes les lignes de la table de gauche, **NULL** à droite si pas de correspondance. **RIGHT JOIN** : l'inverse. **FULL JOIN** : toutes les lignes des deux cotes, avec **NULL** là où il n'y a pas de correspondance. **SELF JOIN** : une table jointe avec elle-même (via deux alias), utile pour des relations hiérarchiques (ex : un employé et son manager, tous deux dans la table **employees**).

Trouver tous les clients sans commande

Tables : `customers(id, name)` et `orders(id, customer_id, amount)`. Ecrivez la requete qui retourne les clients n'ayant jamais passe de commande.

Indice : LEFT JOIN de customers vers orders, puis filtrez sur les lignes ou la cle étrangere du cote orders est NULL.

```
1 SELECT c.id, c.name
2 FROM customers c
3 LEFT JOIN orders o ON o.customer_id = c.id
4 WHERE o.id IS NULL;
```

1.4 Window Functions**A retenir Pourquoi c'est quasi obligatoire en 2026**

Les window functions permettent de calculer une valeur "sur une fenetre de lignes liees" sans effondrer le resultat comme le ferait un `GROUP BY` – chaque ligne d'origine est conservee. C'est l'outil numero un pour le classement, les comparaisons ligne-a-ligne (mois precedent, employe suivant) et la deduplication en SQL pur.

Q.

Quelle est la difference entre `ROW_NUMBER()`, `RANK()` et `DENSE_RANK()` ?

Reponse attendue

`ROW_NUMBER()` attribue un numero unique et strictement croissant a chaque ligne, meme en cas d'egalite (aucun doublon de rang). `RANK()` donne le meme rang aux valeurs ex-aequo mais *saute* les rangs suivants (1, 2, 2, 4). `DENSE_RANK()` donne aussi le meme rang aux ex-aequo mais *sans sauter* de rang (1, 2, 2, 3).

Q.

A quoi servent `LEAD()`, `LAG()` et `NTILE()` ?

Reponse attendue

`LAG(col, n)` recupere la valeur de la colonne `n` lignes avant la ligne courante (dans un ordre donne), `LEAD(col, n)` fait l'inverse (`n` lignes apres) – tres utile pour comparer un mois au mois precedent sans self-join. `NTILE(n)` divise le jeu de resultats en `n` groupes de taille a peu pres egale (ex : `NTILE(4)` pour des quartiles).

Trouver le deuxieme salaire le plus eleve

Table `employees(id, name, salary)`. Ecrivez une requete qui retourne le deuxieme salaire le plus eleve, sans utiliser `LIMIT ... OFFSET` (pensez window function).

```
1 SELECT salary
2 FROM (
3     SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
4     FROM employees
5 ) ranked
6 WHERE rnk = 2;
```

Piege classique

Utiliser `ROW_NUMBER()` au lieu de `DENSE_RANK()` pour "le 2e salaire le plus eleve" est un piege frequent : si les deux salaires les plus hauts sont identiques (ex : 90000, 90000, 85000), `ROW_NUMBER()` renverra 90000 comme "2e" alors que la reponse metier attendue est generealement 85000 – `DENSE_RANK()` gere correctement les ex-aequo.

1.5 CTE et sous-requetes

Q.

A quoi sert une CTE (`WITH ... AS`) et quel est son interet par rapport a une sous-requete imbriquee ?

Reponse attendue

Une CTE (Common Table Expression) nomme un resultat intermediaire pour le reutiliser dans la requete principale, ameliorant fortement la lisibilite par rapport a des sous-requetes imbriquees profondement. Elle permet aussi la recursivite (`WITH RECURSIVE`), utile pour parcourir des hierarchies (ex : organigramme). Contrairement a une vue, elle n'existe que le temps de la requete.

Q.

Quelle est la difference entre une sous-requete correlee (*correlated subquery*) et une sous-requete imbriquee classique ?

Reponse attendue

Une sous-requete imbriquee classique est independante : elle s'execute une seule fois, son resultat est ensuite utilise par la requete externe. Une sous-requete **correlee** reference une colonne de la requete externe et doit donc, conceptuellement, se re-executer pour chaque ligne de la requete externe (ex : `WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.id)`) – souvent plus couteuse si le moteur ne l'optimise pas en jointure.

1.6 Optimisation SQL

Q.

Quelle est la difference entre un index clustered et non-clustered ?

Reponse attendue

Un index **clustered** determine l'ordre physique de stockage des lignes sur le disque – il ne peut y en avoir qu'un seul par table (souvent la cle primaire). Un index **non-clustered** est une structure separee qui pointe vers les lignes reelles : on peut en avoir plusieurs par table, mais chaque lookup implique potentiellement un acces supplementaire ("bookmark lookup") pour recuperer les colonnes non couvertes par l'index.

Q.

A quoi sert un `EXPLAIN` / plan d'execution et que cherchez-vous dedans en premier ?

Reponse attendue

EXPLAIN (ou **EXPLAIN ANALYZE** pour executer reellement la requete et obtenir les temps mesures) montre la strategie choisie par l'optimiseur : quels index sont utilises, l'ordre des jointures, et surtout les *full table scans* sur de grosses tables (souvent le premier signal d'un index manquant) et les operations de tri/hash couteuses en memoire.

Q.

Pourquoi le partitionnement d'une table peut-il ameliorer drastiquement les performances ?

Reponse attendue

Le partitionnement (par date, par region, ...) permet au moteur de ne scanner que les partitions pertinentes pour une requete (*partition pruning*) au lieu de parcourir toute la table. C'est particulierement efficace sur les tables de faits volumineuses filtres systematiquement par date – un pattern omnipresent en data warehousing.

2. Python

Q.

Quelle est la difference entre une `list` et un `tuple` ?

Reponse attendue

Une `list` est mutable (on peut ajouter/modifier/supprimer des elements apres creation), un `tuple` est immuable. Consequence pratique : un `tuple` est hashable (si ses elements le sont) et peut donc servir de cle de dictionnaire ou d'element d'un `set`, contrairement a une `list`.

Q.

Quelle est la difference entre un `dict` et un `set` ?

Reponse attendue

Les deux sont adosses a une table de hachage (recherche en $O(1)$ moyen). Un `dict` stocke des paires cle-valeur ; un `set` stocke uniquement des valeurs uniques (equivalent aux clees d'un `dict` sans les valeurs), utile pour tester l'appartenance ou dedupliquer rapidement.

Q.

Qu'est-ce qu'un generateur (`yield`) et pourquoi est-ce essentiel en ingenierie de donnees ?

Reponse attendue

Un generateur produit ses valeurs une par une, a la demande (*lazy evaluation*), au lieu de construire toute la collection en memoire d'un coup. C'est crucial en data engineering pour traiter des fichiers ou flux trop volumineux pour tenir en RAM (ex : lire un CSV de plusieurs Go ligne par ligne, streamer des enregistrements Kafka).

```
1 def lire_gros_fichier(path):
2     with open(path) as f:
3         for ligne in f:                # une ligne a la fois, pas tout le
4             fichier                    fichier
5             yield ligne.strip()
6
7 # consomme sans jamais charger tout le fichier en memoire
8 for ligne in lire_gros_fichier("logs.csv"):
9     process(ligne)
```

Q.

Quelle est la difference entre un itérateur et un itérable ?

Reponse attendue

Un **itérable** est un objet qui implemente `__iter__()` et peut donc produire un itérateur (ex : une `list`, un `dict`). Un **itérateur** implemente `__next__()` et maintient un etat interne de progression – c'est ce que retourne `iter(un_iterable)`. Un generateur est un cas particulier d'itérateur cree via `yield`.

Q.

Qu'est-ce qu'un decorateur en Python et a quoi cela sert-il typiquement dans un pipeline de donnees ?

Reponse attendue

Un decorateur est une fonction qui enveloppe une autre fonction pour lui ajouter un comportement sans modifier son code (logging, mesure de temps d'execution, retry automatique, cache). En data engineering, on l'utilise couramment pour ajouter de la logique de *retry* autour d'un appel API instable ou pour chronometrer chaque etape d'un pipeline.

```
1 import time
2 from functools import wraps
3
4 def retry(max_attempts=3, delay=2):
5     def decorator(func):
6         @wraps(func)
7         def wrapper(*args, **kwargs):
8             for attempt in range(1, max_attempts + 1):
9                 try:
10                     return func(*args, **kwargs)
11                 except Exception as e:
12                     if attempt == max_attempts:
13                         raise
14                     time.sleep(delay)
15             return wrapper
16     return decorator
17
18 @retry(max_attempts=3)
19 def fetch_api_data(url):
20     ...
```

Q.

Qu'est-ce qu'un *context manager* (*with*) et pourquoi est-il utile pour gerer des connexions ?

Reponse attendue

Un context manager (implementant `__enter__`/`__exit__`, ou via `@contextmanager`) garantit qu'une ressource est correctement liberee (fichier ferme, connexion base de donnees fermee, transaction commit/rollback) meme si une exception survient dans le bloc `with`, sans avoir a ecrire un `try/finally` manuel a chaque usage.

Q.

Quelle est la difference entre une *list comprehension* et un generateur (`(x for x in ...)`) ?

Reponse attendue

Une list comprehension `[x for x in ...]` construit immediatement toute la liste en memoire. Une expression generatrice `(x for x in ...)` (memes crochets remplaces par des parentheses) cree un objet paresseux qui produit les valeurs a la demande – a privilegier des que le resultat est volumineux ou consomme une seule fois (ex : en argument de `sum()`).

Q.

A quoi servent `map()`, `filter()`, `zip()` et `enumerate()` ?

Reponse attendue

`map(f, iterable)` applique `f` a chaque element. `filter(pred, iterable)` garde les elements verifiant le predicat. `zip(a, b)` combine deux iterables element par element en paires (s'arrete au plus court). `enumerate(iterable)` associe chaque element a son index, evitant de gerer un compteur manuel dans une boucle.

3. ETL / ELT

Q.

Qu'est-ce qu'un pipeline ETL ?

Reponse attendue

ETL signifie **Extract** (recuperer les donnees depuis une ou plusieurs sources : API, base, fichiers), **Transform** (nettoyer, agreger, joindre, appliquer des regles metier – generalement hors de l'entrepot cible, sur un moteur de calcul dedie), **Load** (charger le resultat transforme dans la destination finale, ex : un data warehouse).

Q.

Quelle est la difference entre ETL et ELT, et pourquoi ELT est-il privilegie avec Snowflake ou BigQuery ?

Reponse attendue

En **ELT**, on charge d'abord les donnees brutes dans l'entrepot cible, puis on transforme *a l'interieur* de celui-ci (typiquement avec dbt et du SQL). L'ordre ETL transforme *avant* le chargement, sur un moteur externe. ELT est devenu dominant car les entrepots cloud modernes (Snowflake, BigQuery) offrent une puissance de calcul elastique et bon marche, capable d'absorber des transformations lourdes directement en SQL – evitant de maintenir un moteur de transformation separe et permettant de conserver les donnees brutes pour re-transformer facilement si la logique metier change.

Q.

Dans un pipeline moderne (API → Kafka → Spark → Delta Lake → dbt → Power BI), quel est le role de chaque etage ?

Reponse attendue

API : source de donnees. **Kafka** : ingestion et decouplage temps reel (buffer resilient). **Spark** : traitement/transformation a grande echelle (batch ou streaming). **Delta Lake** : stockage versionne et fiable (ACID sur un data lake) organise en couches Bronze/Silver/Gold. **dbt** : transformations SQL declaratives et testees dans l'entrepot. **Power BI** : consommation/visualisation finale par les utilisateurs metier.

Q.

Quelle est la difference entre l'ingestion, la transformation, l'orchestration et le monitoring d'un pipeline ?

Reponse attendue

Ingestion : faire entrer la donnee dans le systeme (batch ou streaming). **Transformation** : la nettoyer/structurer/enrichir. **Orchestration** : sequencer et declencher les differentes etapes dans le bon ordre, avec gestion des dependances et des reprises (ex : Airflow). **Monitoring** : surveiller que chaque etape s'execute correctement, dans les delais (SLA), et alerter en cas d'anomalie.

Q.

Qu'est-ce qu'un *backfill* et pourquoi est-ce une operation delicate ?

Reponse attendue

Un backfill consiste a re-executer un pipeline pour une periode passee (ex : recalculer les 30 derniers jours apres correction d'un bug de logique metier). C'est delicat car cela peut generer une charge massive sur les systemes sources/cibles, entrer en conflit avec les executions courantes, et necessite generalement que le pipeline soit **idempotent** pour eviter de dupliquer des donnees deja presentes.

4. Data Modeling

A retenir L'état d'esprit attendu

Les interviewers veulent voir un raisonnement en termes de **grain** (quel est le niveau de détail d'une ligne?), de **faits** (mesures numeriques) et de **dimensions** (axes d'analyse) – pas juste "je cree des tables SQL normalisees".

Q.

Quelle est la difference entre une table de faits (*fact table*) et une table de dimension ?

Reponse attendue

Une **table de faits** contient les mesures quantitatives d'un evenement metier (montant d'une vente, quantite, duree) a un grain precis, plus des clees etrangeres vers les dimensions concernees. Une **table de dimension** contient les attributs descriptifs qui donnent du contexte a ces mesures (nom du client, categorie de produit, date calendaire) – generalement peu volumineuse mais large en nombre de colonnes.

Q.

Quelle est la difference entre un schema en etoile (*star schema*) et un schema en flocon (*snowflake schema*) ?

Reponse attendue

Le **star schema** a une table de faits centrale reliee directement a des tables de dimensions denormalisees (chaque dimension est une seule table plate) – simple, rapide a interroger (moins de jointures), au prix d'une certaine redondance. Le **snowflake schema** normalise davantage les dimensions en sous-tables liees (ex : `dim_product` → `dim_category` → `dim_department`), reduisant la redondance mais augmentant le nombre de jointures necessaires pour une requete.

Q.

Qu'est-ce qu'une SCD (*Slowly Changing Dimension*) et quelle est la difference entre les types 1, 2 et 3 ?

Reponse attendue

Une SCD gere l'evolution dans le temps des attributs d'une dimension (ex : un client change d'adresse). **Type 1** : on efface simplement l'ancienne valeur – pas d'historique conserve. **Type 2** : on insere une nouvelle ligne avec une nouvelle version de la dimension, en marquant les dates de validite (`valid_from/valid_to`) ou un flag `is_current` – l'historique complet est conserve, c'est le plus utilise en pratique. **Type 3** : on ajoute une colonne supplementaire pour garder uniquement l'ancienne *et* la nouvelle valeur (historique limite a un seul changement).

Modeliser une plateforme e-commerce

Concevez le modele de donnees (star schema) d'une plateforme e-commerce permettant d'analyser les ventes par produit, par client et dans le temps.

Reflechissez d'abord au grain de la table de faits (une ligne = une commande ? une ligne d'article commande ?), puis identifiez les dimensions necessaires (`dim_product`, `dim_customer`,

dim_date, dim_store) avant d'écrire le moindre DDL.

5. Apache Spark / PySpark

Q.

Quelle est la difference entre RDD, DataFrame et Dataset dans Spark ?

Reponse attendue

RDD (Resilient Distributed Dataset) est l'abstraction bas niveau originelle : une collection distribuee d'objets Java/Python sans schema, sans optimisation automatique. **DataFrame** ajoute un schema (colonnes typees) et beneficie de l'optimiseur Catalyst et du moteur d'execution Tungsten – c'est l'API standard aujourd'hui, y compris en PySpark. **Dataset** (uniquement Scala/Java) combine la securite de type a la compilation des RDD avec les optimisations des DataFrames – n'existe pas en Python, ou les DataFrames sont deja dynamiquement types.

Q.

Quelle est la difference entre une transformation et une action dans Spark ?

Reponse attendue

Une **transformation** (`map`, `filter`, `groupBy`, `join`) est *paresseuse* (lazy) : elle construit un plan d'execution (DAG) sans rien calculer immediatement. Une **action** (`collect`, `count`, `take`, `show`) declenche reellement l'execution du DAG accumule. Cette paresse permet a Spark d'optimiser globalement la chaine de transformations avant de l'executer.

Piege classique

Appeler `.collect()` sur un DataFrame trop volumineux rapatrie *toutes* les donnees vers le driver (noeud unique), pouvant provoquer un `OutOfMemoryError` – un piege frequemment mentionne en entretien. Preferer `.show(n)` pour inspecter un echantillon, ou ecrire le resultat directement vers un stockage distribue.

Q.

Quelle est la difference entre `repartition()` et `coalesce()` ?

Reponse attendue

`repartition(n)` redistribue les donnees en `n` nouvelles partitions via un *shuffle* complet (couteux mais equilibre) – utile pour augmenter le parallelisme ou corriger un skew. `coalesce(n)` reduit le nombre de partitions en fusionnant des partitions existantes sur les memes noeuds, *sans shuffle complet* (plus rapide), mais ne peut pas augmenter le nombre de partitions ni garantir un equilibrage parfait.

Q.

Pourquoi le *shuffle* est-il couteux dans Spark ?

Reponse attendue

Un shuffle redistribue les donnees entre les partitions/executeurs a travers le reseau (ex : pour un `groupBy` ou un `join` sur une cle), impliquant serialisation, ecriture disque intermediaire,

transfert reseau et deserialisation – des ordres de grandeur plus lent qu'un traitement local a une partition. Minimiser les shuffles (via un design de partitionnement adapte, des broadcast joins, ou en filtrant tot) est l'un des leviers d'optimisation les plus importants.

Q.

Comment accelerer un job Spark lent ? Citez les principaux leviers.

Reponse attendue

Broadcast join : diffuser une petite table sur tous les executeurs pour eviter un shuffle lors d'une jointure avec une grosse table. **Cache/persist** : mettre en memoire un DataFrame reutilise plusieurs fois pour eviter de le recalculer. **Partitionnement adapte** : eviter le *data skew* (une partition beaucoup plus grosse que les autres qui devient le goulot d'etranglement) en repartitionnant sur une cle bien distribuee. **Adaptive Query Execution (AQE)** : Spark ajuste dynamiquement le plan d'execution (nombre de partitions post-shuffle, strategie de jointure) en fonction des statistiques reelles observees a l'execution.

```
1 from pyspark.sql import functions as F
2
3 # Broadcast join : evite un shuffle si dim_product est petite
4 result = fact_sales.join(
5     F.broadcast(dim_product), on="product_id", how="left"
6 )
7
8 # Cache d'un DataFrame reutilise plusieurs fois
9 df_clean = df.filter(F.col("amount") > 0).cache()
10 df_clean.count() # declenche le calcul et peuple le cache
```

Q.

Qu'est-ce que le *data skew* et comment le detecter/corriger ?

Reponse attendue

Le skew survient quand une ou quelques cles concentrent une part disproportionnee des donnees (ex : 80% des commandes viennent d'un seul `customer_id` de test), faisant qu'une seule partition/tache met beaucoup plus de temps que les autres et bloque tout le job. On le detecte via l'UI Spark (une tache largement plus longue que les autres dans un stage). On le corrige par *salting* (ajouter un suffixe aleatoire a la cle skewee pour repartir la charge) ou en laissant l'AQE le gerer automatiquement dans les versions recentes de Spark.

6. Kafka

Q.

Definissez Producer, Consumer, Topic, Partition, Broker et Consumer Group.

Reponse attendue

Producer : application qui publie des messages. **Consumer** : application qui lit des messages. **Topic** : flux logique nomme de messages, divise en **partitions** pour permettre le parallelisme (chaque partition est un log ordonne et immuable). **Broker** : serveur Kafka qui stocke les partitions et sert les requetes. **Consumer Group** : ensemble de consumers qui se repartissent les partitions d'un topic, chaque partition n'etant lue que par un seul consumer du groupe a la fois (garantit le parallelisme sans double lecture au sein du meme groupe).

Q.

Qu'est-ce qu'un *offset* dans Kafka ?

Reponse attendue

C'est un identifiant sequentiel unique attribue a chaque message au sein d'une partition, representant sa position. Le consumer group memorise (commit) le dernier offset traite par partition, ce qui lui permet de reprendre exactement ou il s'est arrete en cas de redemarrage, sans perdre ni dupliquer de messages (si bien gere).

Q.

Pourquoi Kafka est-il si performant ?

Reponse attendue

Plusieurs raisons : ecriture sequentielle sur disque (bien plus rapide que des ecritures aleatoires, meme sur disque classique), usage intensif du *zero-copy* (transfert de donnees directement du cache disque vers le reseau sans passer par l'espace utilisateur de la JVM), traitement par lots (*batching*) des messages pour amortir le cout reseau, et parallelisme horizontal via les partitions.

Q.

Pourquoi utiliser plusieurs partitions sur un topic, et comment garantir l'ordre des messages ?

Reponse attendue

Plusieurs partitions permettent le parallelisme : plusieurs consumers d'un meme groupe peuvent lire simultanement des partitions differentes. Le prix a payer : Kafka ne garantit l'ordre des messages *qu'a l'interieur d'une meme partition*, pas entre partitions. Pour garantir l'ordre des messages relatifs a une meme entite (ex : tous les evenements d'un `user_id`), il faut utiliser cette entite comme **cle de partitionnement** : Kafka route alors deterministement tous les messages de cette cle vers la meme partition.

Q.

Quelle est la difference entre *at-most-once*, *at-least-once* et *exactly-once* ?

Reponse attendue

At-most-once : un message peut etre perdu mais jamais duplique (on commit l'offset avant de traiter – si le traitement echoue, le message est perdu). **At-least-once** : un message n'est jamais perdu mais peut etre traite plusieurs fois (on commit l'offset apres traitement – si le commit echoue apres un traitement reussi, le message sera retraite); le consumer doit alors etre idempotent. **Exactly-once** : garantie la plus forte, chaque message est traite exactement une fois – obtenue via les transactions Kafka (producer transactionnel + consumer en lecture "read_committed") ou par idempotence applicative combinee a at-least-once.

7. Airflow

Q.

Definissez DAG, Task, Operator, Scheduler et Executor.

Reponse attendue

DAG (Directed Acyclic Graph) : la definition d'un pipeline sous forme de graphe de dependances sans cycle. **Task** : une unite de travail concrete dans le DAG (une instance d'un Operator). **Operator** : un template reutilisable definissant *quoi* faire (ex : `PythonOperator`, `BashOperator`, `PostgresOperator`). **Scheduler** : le processus qui decide quand declencher chaque DAG/task selon sa planification et ses dependances. **Executor** : le mecanisme qui execute reellement les tasks (ex : `LocalExecutor`, `CeleryExecutor`, `KubernetesExecutor` pour distribuer l'execution).

Q.

A quoi sert XCom dans Airflow ?

Reponse attendue

XCom (*cross-communication*) permet a des tasks d'echanger de petites quantites de donnees entre elles (ex : un ID genere par une task, consomme par la suivante). Ce n'est pas concu pour transporter de gros volumes de donnees (celles-ci doivent transiter par un stockage externe comme S3, la task ne passant que la reference/chemin via XCom).

Q.

Quelle est la difference entre un Sensor et un Operator classique ?

Reponse attendue

Un **Sensor** est un type special d'Operator qui attend qu'une condition externe soit remplie avant de laisser le DAG continuer (ex : attendre qu'un fichier arrive sur S3, qu'une partition soit disponible). Il "poll" periodiquement jusqu'a succes ou timeout, contrairement a un Operator classique qui execute une action et se termine.

Q.

Quelle est la difference entre `retry`, `backfill` et `catchup` dans Airflow ?

Reponse attendue

Retry : relance automatique d'une task ayant echoue, selon une politique configuree (`retries`, `retry_delay`). **Backfill** : execution manuelle du DAG pour des dates passees specifiques (souvent apres correction d'un bug). **Catchup** : comportement par default d'Airflow qui, si le DAG a ete cree ou reactive avec une `start_date` passee, va automatiquement executer toutes les executions "manquees" depuis cette date – souvent desactive explicitement (`catchup=False`) pour eviter une rafale d'executions non desirees.

Creer un DAG Extract → Transform → Load

Ecrivez un DAG Airflow simple avec 3 tasks sequentielles (extract, transform, load) utilisant `PythonOperator`, avec une politique de retry de 2 tentatives espaces de 5 minutes.

```
1 from airflow import DAG
2 from airflow.operators.python import PythonOperator
3 from datetime import datetime, timedelta
4
5 default_args = {
6     "retries": 2,
7     "retry_delay": timedelta(minutes=5),
8 }
9
10 with DAG(
11     dag_id="etl_ventes",
12     schedule="@daily",
13     start_date=datetime(2026, 1, 1),
14     catchup=False,
15     default_args=default_args,
16 ) as dag:
17
18     extract = PythonOperator(task_id="extract", python_callable=
19         extract_data)
20     transform = PythonOperator(task_id="transform", python_callable=
21         transform_data)
22     load = PythonOperator(task_id="load", python_callable=load_data)
23
24     extract >> transform >> load
```

8. Data Warehouse et formats de table

Q.

Citez les principaux entrepôts de données cloud et une caractéristique distinctive de chacun.

Reponse attendue

Snowflake : separation totale du stockage et du calcul, multi-cloud, tres simple a administrer. **Databricks** : plateforme unifiée data engineering + ML autour de Spark et Delta Lake (le "Lakehouse"). **BigQuery** : serverless, facturation a la requete scannee, tres integre a l'écosysteme GCP. **Redshift** : entrepot AWS historique, base sur PostgreSQL, architecture en clusters. **Synapse** : offre analytique unifiée d'Azure combinant entrepot et Big Data.

Q.

Quelle est la difference entre le partitionnement et le clustering d'une table dans un entrepot cloud ?

Reponse attendue

Le **partitionnement** divise physiquement la table en segments distincts selon une colonne (souvent une date), permettant d'éliminer des partitions entieres avant meme de scanner (*pruning*). Le **clustering** (ou "tri physique") organise les données *a l'interieur* des partitions (ou de la table) selon une ou plusieurs colonnes frequemment filtrees, permettant au moteur de sauter des blocs de données non pertinents sans creer autant de partitions distinctes qu'il y aurait de valeurs.

Q.

Pourquoi utiliser Delta Lake (ou un format similaire) plutot que des fichiers Parquet bruts sur un data lake ?

Reponse attendue

Delta Lake ajoute par-dessus Parquet une couche de *transaction log* (fichiers JSON versionnes) qui apporte : transactions **ACID** (evite les etats incoherents en cas d'écriture concurrente ou d'échec en cours de route), **time travel** (interroger une version anterieure de la table), gestion fiable des **schema evolution/enforcement**, et compaction/optimisation facilitees (**OPTIMIZE**, **VACUUM**) – des garanties qu'un simple repertoire de fichiers Parquet n'offre pas nativement.

Q.

Quelle est la difference entre Delta Lake, Apache Iceberg et Apache Hudi ?

Reponse attendue

Les trois resolvent le meme probleme (apporter des transactions ACID et du versioning a un data lake) avec des philosophies proches mais des ecosystemes differents. **Delta Lake** est historiquement le plus lie a Databricks/Spark. **Iceberg** met l'accent sur l'interopérabilité multi-moteurs (Spark, Trino, Flink, Snowflake) et une gestion tres fine de l'évolution de schema/partitionnement sans reecrire les données. **Hudi** est particulierement optimise pour

les cas d'usage avec des mises a jour/upserts frequents et le streaming quasi temps-reel (*incremental processing*).

Q.

Qu'est-ce que l'architecture Bronze / Silver / Gold (medaillon) et pourquoi l'adopter ?

Reponse attendue

C'est un pattern d'organisation d'un lakehouse en 3 couches progressives : **Bronze** (donnees brutes, telles qu'ingerees, non modifiees – source de verite pour rejouer les traitements), **Silver** (donnees nettoyees, dedupliquees, jointes – structure exploitable mais encore assez granulaire), **Gold** (donnees agregees et modelisees pour la consommation metier – souvent en star schema). Cette separation permet de rejouer une transformation en cas d'erreur sans re-ingerer depuis la source, et de donner a chaque equipe (data engineers, analystes) le niveau de donnee adapte a son besoin.

9. Cloud (AWS, Azure, GCP)

9.1 AWS

Q.

Quel est le role de S3, Glue, EMR, Athena, Lambda et IAM dans une architecture data AWS ?

Reponse attendue

S3 : stockage objet, socle du data lake. **Glue** : catalogue de metadonnees (Data Catalog) + service ETL serverless (crawlers, jobs Spark managés). **EMR** : clusters Hadoop/Spark managés pour du traitement Big Data a grande echelle. **Athena** : moteur de requete SQL serverless directement sur les fichiers S3 (base sur Presto/Trino), facture a la donnee scannee. **Lambda** : fonctions serverless pour du traitement evenementiel leger (ex : declencher un traitement a l'arrivee d'un fichier S3). **IAM** : gestion des identites et permissions – essentiel pour securiser l'acces aux donnees entre services.

9.2 Azure

Q.

Quel est le role de ADLS, Synapse, Databricks et Event Hub dans une architecture data Azure ?

Reponse attendue

ADLS (Azure Data Lake Storage) : stockage objet hierarchique optimise pour la Big Data. **Synapse** : plateforme analytique unifiee (entrepot + Spark + pipelines). **Databricks** (disponible nativement sur Azure) : plateforme Lakehouse Spark/Delta. **Event Hub** : ingestion d'evenements a tres grande echelle, equivalent Azure de Kafka pour le streaming.

9.3 GCP

Q.

Quel est le role de BigQuery, Pub/Sub, Dataflow et Composer dans une architecture data GCP ?

Reponse attendue

BigQuery : entrepot serverless, coeur analytique de l'ecosysteme GCP. **Pub/Sub** : messagerie evenementielle globale, equivalent GCP de Kafka. **Dataflow** : traitement de donnees batch et streaming unifie, base sur Apache Beam. **Composer** : Airflow manage par Google, pour l'orchestration.

10. System Design Data, Data Quality et Orchestration

10.1 System Design

A retenir Methode pour repondre a un exercice de system design data

Suivez toujours la meme trame : (1) clarifiez les besoins (volume, latence attendue batch vs temps-reel, SLA), (2) proposez une architecture haut niveau couche par couche (ingestion → stockage → traitement → service), (3) justifiez chaque choix technologique par une contrainte identifiee en (1), (4) anticipez les points de defaillance (retard, doublons, panne) et comment le design les gere.

Q.

Dans une architecture "API → Kafka → Spark Streaming → Delta Bronze → Silver → Gold → Power BI", justifiez le choix de Kafka, Spark et Delta.

Reponse attendue

Kafka decouple les producteurs des consommateurs et absorbe les pics de charge sans perdre de donnees (buffer durable et rejouable). **Spark Streaming** traite ces evenements en quasi temps-reel avec un parallelisme horizontal et une tolerance aux pannes native. **Delta Lake** apporte des transactions ACID sur le resultat, permettant des ecritures concurrentes fiables (plusieurs jobs Silver/Gold en parallele) et la possibilite de corriger/rejouer une transformation sans corrompre les donnees en aval.

Q.

Comment gerez-vous les donnees qui arrivent en retard (*late-arriving data*) dans un pipeline streaming ?

Reponse attendue

On definit une fenetre de tolerance (*watermark*) qui accepte les evenements en retard jusqu'a un certain seuil avant de considerer une fenetre temporelle comme definitivement close. Au-dela du watermark, l'evenement en retard est soit rejete, soit route vers une table/traitement dedie pour reconciliation ulterieure. Le compromis est explicite : plus le watermark est large, plus on capture de donnees en retard, mais plus on retarde la disponibilite des resultats.

Q.

Comment rendre un pipeline idempotent ?

Reponse attendue

Un pipeline est idempotent si l'executer plusieurs fois avec les memes donnees d'entree produit toujours le meme resultat final, sans doublons ni effets cumulatifs. Techniques courantes : utiliser des operations MERGE/UPSERT plutot que de simples INSERT (base sur une cle metier), ecrire par ecrasement complet d'une partition cible (*overwrite by partition*) plutot qu'un append, ou dedupliquer explicitement en aval sur un identifiant unique d'evenement avant ecriture finale.

10.2 Data Quality

Q.

Comment detecter des doublons, des valeurs nulles ou des donnees corrompues dans un pipeline ?

Reponse attendue

Doublons : verifier l'unicite d'une cle metier attendue (ex : `COUNT(*)` vs `COUNT(DISTINCT id)`). Valeurs nulles : controler le taux de `NULL` sur les colonnes critiques par rapport a un seuil attendu. Donnees corrompues : valider le schema/types a l'ingestion, verifier des plages de valeurs plausibles (ex : un age negatif, un montant aberrant). En pratique, on automatise ces controles avec des outils comme Great Expectations, dbt tests, ou des assertions custom executees a chaque run du pipeline, avec alerte/blocage si un seuil est depasse.

Q.

Qu'est-ce que le *data drift* et pourquoi le surveiller ?

Reponse attendue

Le data drift est un changement progressif (ou brutal) dans la distribution statistique des donnees au fil du temps par rapport a une baseline de reference (ex : la distribution des ages des utilisateurs change apres le lancement dans un nouveau pays). Non detecte, il peut degrader silencieusement la qualite des analyses ou des modeles ML consommant ces donnees – on le surveille en comparant periodiquement des statistiques (moyenne, distribution, cardinalite) entre la periode courante et une reference.

10.3 Orchestration

Q.

Comment relancez-vous un pipeline en echec et comment gerez-vous les dependances entre taches ?

Reponse attendue

En premier lieu, une politique de **retry** automatique geree par l'orchestrateur (Airflow) absorbe les echecs transitoires (timeout reseau, service momentanement indisponible). Si l'echec persiste, on isole la task en echec (statut visible dans l'UI), on corrige la cause racine, puis on relance uniquement les tasks impactees (et leurs aval) plutot que tout le DAG – possible grace au graphe de dependances explicite qui identifie precisement quelles tasks downstream doivent etre re-executees.

Q.

Comment surveillez-vous les SLA d'un pipeline de donnees ?

Reponse attendue

On definit un delai maximal acceptable entre le declenchement attendu d'un pipeline et sa completion effective (ex : "les donnees Gold doivent etre pretes avant 8h"). Airflow expose des alertes SLA natives (`sla` sur une task), completees par des dashboards de monitoring (temps

d'exécution historique, taux d'échec) et des alertes proactives (Slack, email, PagerDuty) dès qu'un seuil est en passe d'être dépassé plutôt que seulement après coup.

11. Questions sur vos projets

A retenir Pourquoi c'est 90% de l'entretien

Les recruteurs veulent verifier que vous comprenez *pourquoi* vous avez fait chaque choix technique, pas juste que vous savez nommer les outils. Preparez pour chaque techno de votre stack une reponse en 3 temps : (1) le probleme concret que ca resout, (2) pourquoi cette techno plutot qu'une alternative evidente, (3) une limite ou un compromis assume.

Q : Pourquoi Kafka dans votre architecture ?

R : Decoupler les producteurs et consommateurs pour absorber des pics de charge et permettre un traitement asynchrone et rejouable, plutot qu'un couplage synchrone fragile entre services.

Q : Pourquoi Spark plutot qu'un traitement pandas classique ?

R : Besoin de distribuer le calcul sur un volume de donnees depassant la memoire d'une seule machine, avec tolerance aux pannes native et parallelisme horizontal – pandas reste pertinent pour des volumes qui tiennent en memoire sur une machine.

Q : Pourquoi Airflow plutot qu'un simple cron ?

R : Besoin de gerer des dependances explicites entre taches, des reprises automatiques sur echec, une visibilite centralisee (UI) sur l'etat des pipelines, et un historique d'executions – un cron ne offre aucune de ces garanties nativement.

Q : Pourquoi Delta Lake / Iceberg plutot que des fichiers Parquet bruts ?

R : Besoin de transactions ACID sur des ecritures concurrentes, de time travel pour deboguer/auditer, et d'une gestion fiable de l'evolution de schema dans le temps.

Q : Pourquoi dbt pour les transformations ?

R : Transformations exprimees en SQL declaratif, versionnees, testables (dbt tests) et documentees, avec une gestion automatique du graphe de dependances entre modeles – evite les scripts de transformation ad hoc difficiles a maintenir.

Q : Pourquoi MinIO ?

R : Stockage objet compatible S3 auto-heberge, utile pour du developpement local ou un environnement on-premise sans dependre d'un cloud provider, tout en gardant la meme API que S3 en production.

Q : Pourquoi PostgreSQL comme base operationnelle ?

R : Base relationnelle mature avec des garanties transactionnelles fortes (ACID), un bon support des extensions (ex : PostGIS, pg_cron) et un ecosysteme d'outils tres riche – a adapter avec les vraies raisons de votre projet.

Q : Pourquoi Docker pour ce projet ?

R : Garantir un environnement reproductible entre les machines de l'equipe et faciliter l'orchestration de plusieurs services (Kafka, Spark, base de donnees) localement via Docker Compose avant le deploiement.

Q : Pourquoi MLflow si votre projet integre du ML ?

R : Tracer systematiquement les experimentations (parametres, metriques, artefacts de modele) et gerer le cycle de vie des modeles (registry, versioning, deploiement) plutot que de garder des notes eparses sur les runs d'entrainement.

Q.

Quel est le plus gros probleme que vous avez rencontre sur un pipeline en production et comment l'avez-vous resolu ?

Reponse attendue

Structurez la reponse en methode : symptome observe (ex : job Spark 10x plus lent, Kafka en retard), hypotheses testees (via les metriques/logs disponibles), cause racine identifiee (ex : data skew, absence d'index, saturation du consumer group), correction apportee, et ce que vous en avez tire pour eviter la recidive (alerte ajoutee, test de non-regression, documentation).

12. Scenarios de production

A retenir Ce que l'évaluateur cherche vraiment

Ces questions ne testent pas votre connaissance d'un outil précis mais votre **demarche de diagnostic** : commencer par observer (logs, metriques) avant d'agir, formuler des hypotheses avant de conclure, et distinguer une solution "pour reparer maintenant" d'une solution "pour que ca ne se reproduise plus".

Q.

Un pipeline est en echec a 3h du matin. Que faites-vous ?

Reponse attendue

D'abord evaluer l'impact (quelles donnees/consommateurs en aval sont affectes, y a-t-il un SLA menace ?). Consulter les logs de la task en echec pour identifier la cause immediate (erreur transitoire vs structurelle). Si transitoire (timeout, service source momentanement indisponible), relancer via le retry ou manuellement. Si structurelle (schema change, donnee corrompue), isoler le pipeline pour eviter de propager des donnees fausses en aval, puis investiguer a tete reposee – ne pas forcer une relance aveugle qui pourrait aggraver la situation.

Q.

Le job Spark est 10 fois plus lent qu'hier. Quelles pistes explorez-vous ?

Reponse attendue

Verifier d'abord si le *volume de donnees* en entree a significativement augmente (cause la plus frequente et la plus benigne). Ensuite, consulter l'UI Spark pour reperer un *data skew* (une tache anormalement longue dans un stage) ou un changement de plan d'execution (ex : bascule d'un broadcast join vers un shuffle join si une table a grossi au-dela du seuil de broadcast). Verifier aussi les ressources du cluster (executeurs disponibles, memoire) et si une modification recente du code a introduit une operation couteuse (ex : un `collect()` ou un UDF non vectorise).

Q.

Kafka accumule du retard (*consumer lag*). Comment reagissez-vous ?

Reponse attendue

Verifier si le *debit de production* a augmente (pic legitime) ou si le *consumer* est devenu plus lent (bug, dependance externe lente, ressources insuffisantes). Solutions : augmenter le nombre de consumers dans le groupe jusqu'a concurrence du nombre de partitions (au-dela, des consumers restent inactifs), optimiser le traitement par message, ou augmenter le nombre de partitions du topic (operation qui casse cependant l'ordre garanti au sein de l'ancien partitionnement, a faire avec precaution).

Q.

Une table contient des doublons. Comment diagnostiquez-vous et corrigez-vous le probleme ?

Reponse attendue

Diagnostiquer d'abord la cause : un pipeline non idempotent relance plusieurs fois sur les memes donnees, une jointure "fan-out" non anticipee (ex : jointure sur une cle non unique cote droit), ou une absence de deduplication a l'ingestion. Corriger a la source (rendre le pipeline idempotent via MERGE/UPSERT) plutot que de seulement nettoyer les doublons existants en aval, sous peine de voir le probleme se reproduire a chaque run.

Q.

Une donnee arrive avec 24h de retard. Comment votre pipeline la gere-t-il ?

Reponse attendue

Cela depend de la fenetre de tolerance (watermark) definie en amont : si le retard est dans la fenetre acceptee, la donnee est integree normalement lors du prochain traitement de sa periode. Si le pipeline traite deja par partitions fermees (ex : Gold deja calcule pour la veille), il faut soit re-declencher un backfill cible sur la partition concernee, soit accepter que cette donnee soit comptabilisee dans une periode ulterieure – un compromis metier a clarifier en amont avec les parties prenantes, pas a decider seul dans l'urgence.

Q.

Une partition de donnees est corrompue. Quelle est votre demarche ?

Reponse attendue

Isoler immediatement la partition corrompue pour eviter qu'elle ne soit consomsee en aval (ex : la marquer invalide plutot que de la supprimer directement, pour investigation). Identifier si une version anterieure fiable existe (time travel avec Delta Lake/Iceberg permet de revenir a l'etat precedent). Si la source brute (couche Bronze) est intacte, on peut simplement retraiter cette partition specifique depuis la source – d'ou l'interet de toujours conserver les donnees brutes non modifiees.

Q.

Un DAG Airflow reste bloque indefiniment. Que verifiez-vous ?

Reponse attendue

Verifier en priorite si une task est bloquee sur un **Sensor** en attente d'une condition qui ne se realisera jamais (fichier attendu jamais arrive, dependance externe en panne) – cause tres frequente. Verifier ensuite la disponibilite des *slots* de l'executor (pool satures, trop de DAGs concurrents) et les logs du scheduler pour ecarter un probleme d'infrastructure sous-jacent plutot que du DAG lui-meme.